

e) Scraping de reseñas de entrega de órdenes usando Playwright

Objetivo: descargar **todas** las reseñas sobre la entrega de los pedidos (`tipo = "post_compra"`) y guardarlas en `data/resenas_entrega.csv` .

Estas reseñas aparecen incrustadas dentro del listado paginado de órdenes (`/ordenes`), una por cada pedido que las tiene. A diferencia de los ejercicios b–d (que usan `requests` + BeautifulSoup sobre el HTML ya renderizado por el servidor), aquí usamos **Playwright** para automatizar un navegador real: se abre cada página del listado en un navegador headless y se localizan los elementos con su propia API de *locators*, en vez de descargar y parsear el HTML manualmente.

Esta técnica es la que se necesitaría en sitios donde el contenido se genera dinámicamente en el cliente (JavaScript puro, SPA); aquí la aplicamos igualmente sobre la tienda virtual para practicar el flujo de trabajo de Playwright: lanzar el navegador, navegar, esperar a que el contenido esté listo y extraer datos con locators.

```
import csv
import os
from playwright.async_api import async_playwright

BASE_URL = "http://localhost:3000"
USER_AGENT = "MineriaWeb-2026-2/1.0 (+scraper-tienda-virtual)"
```

Recorrido paginado con Playwright

Igual que en los ejercicios c) y d), detenemos la paginación cuando el enlace **Siguiente** del componente de paginación deja de ser un `<a>` (se vuelve `<span>` deshabilitado). Con Playwright esto se resuelve contando cuántos elementos coinciden con el selector `nav[aria-label="Paginacion"] a:has-text("Siguiente")` .

```
resenas_entrega = []

playwright = await async_playwright().start()
navegador = await playwright.chromium.launch(headless=True)
pagina = await navegador.new_page(user_agent=USER_AGENT)

numero_pagina = 1
while True:
    await pagina.goto(f"{BASE_URL}/ordenes?page={numero_pagina}",
wait_until="networkidle")

    ordenes = pagina.locator("article[data-orden-id]")
    total_ordenes = await ordenes.count()
    if total_ordenes == 0:
        break
```

```

for i in range(total_ordenes):
    orden = ordenes.nth(i)
    orden_id = await orden.get_attribute("data-orden-id")

    resenas_de_la_orden = orden.locator("li[data-tipo='post_compra']")
    total_resenas_orden = await resenas_de_la_orden.count()
    for j in range(total_resenas_orden):
        resena = resenas_de_la_orden.nth(j)
        resenas_entrega.append({
            "id": await resena.get_attribute("data-comentario-id"),
            "orden_id": orden_id,
            "cliente_id": await resena.get_attribute("data-cliente-id"),
            "producto_id": await resena.get_attribute("data-producto-id"),
            "calificacion": await resena.get_attribute("data-calificacion"),
            "fecha": await resena.get_attribute("data-fecha"),
            "texto": await resena.locator("[itemprop='reviewBody']").inner_text(),
        })

    print(f"Pagina {numero_pagina}: {total_ordenes} ordenes revisadas")

    siguiente = pagina.locator('nav[aria-label="Paginacion"] a:has-text("Siguiente")')
    if await siguiente.count() == 0:
        break
    numero_pagina += 1

await navegador.close()
await playwright.stop()

print(f"\nTotal de resenas de entrega scrapeadas: {len(resenas_entrega)} en {numero_pagina} paginas de ordenes")

```

Total de resenas de entrega scrapeadas: 0 en 1 paginas de ordenes

Guardar los datos en data/resenas_entrega.csv

```

DATA_DIR = os.path.join(os.getcwd(), "..", "data")
os.makedirs(DATA_DIR, exist_ok=True)
OUTPUT_PATH = os.path.join(DATA_DIR, "resenas_entrega.csv")

with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
    writer = csv.DictWriter(archivo, fieldnames=resenas_entrega[0].keys())
    writer.writeheader()
    writer.writerows(resenas_entrega)

print(f"Se guardaron {len(resenas_entrega)} resenas de entrega en {OUTPUT_PATH}")

```

IndexError

Traceback (most recent call last)

Cell In[3], line 6

```
2 os.makedirs(DATA_DIR, exist_ok=True)
3 OUTPUT_PATH = os.path.join(DATA_DIR, "resenas_entrega.csv")
4
5 with open(OUTPUT_PATH, mode="w", newline="", encoding="utf-8") as archivo:
----> 6     writer = csv.DictWriter(archivo, fieldnames=resenas_entrega[0].keys())
7     writer.writeheader()
8     writer.writerows(resenas_entrega)
9
```

IndexError: list index out of range